001 -- Data Acquisition

Author: Wayne Kirk Schmidt
Email: wayne.kirk.schmidt@gmail.com

Purpose

`001_download.ipynb` is the first stage of the cryptocurrency statistical arbitrage research pipeline.

This notebook retrieves daily OHLCV market data from Binance and constructs aligned price series and per-asset event panels for use in all downstream stages.

Responsibilities

- Define the cryptocurrency universe
- Download daily price and volume data from Binance
- Align time series across all assets into a single price matrix
- Construct per-asset lead-lag event panels

Outputs

Artifacts written to `output/001_download/` :

File	Description
`PRICES.pkl`	Long-format DataFrame of daily OHLCV for all assets
`<COIN>.event_panel.pkl`	Per-asset panel of reference price and lagged target closes
`manifest.pkl`	Updated pipeline manifest with artifact paths and timestamps

Pipeline Position

```
001_download  <- you are here
     -
002_enrich    -> feature engineering and z-scores
     -
003_analysis  -> shock detection and cross-asset analysis
     -
004_strategy  -> signal construction
     -
005_backtest  -> execution and performance evaluation
```

1. Imports and Environment Setup

Standard library imports. `binance-python` is used to access the Binance public REST API without authentication.

In [1]:
```
import pandas as pd
import numpy as np
import pickle
from binance.client import Client
import matplotlib.pyplot as plt
import math
from pathlib import Path
from datetime import datetime, timedelta, UTC
```

2. Environment Configuration

Defines the asset universe, pipeline parameters, and output paths. All downstream notebooks inherit this same universe and date range via the manifest.

Key parameters:

- `startdate` -- earliest date for downloaded price history
- `universe` -- the 9 USDT-margined pairs under study
- `observation_window_length` -- number of lag days captured in each event panel row
- `holding_period` -- assumed trade holding period in days

In [2]:
```
startdate = "2023-01-01"
trading_days = 252
frequency = "1d"

universe = [
    "BTCUSDT",   # Bitcoin
    "ETHUSDT",   # Ethereum
    "BNBUSDT",   # Binance Coin
    "SOLUSDT",   # Solana
    "XRPUSDT",   # Ripple
    "ADAUSDT",   # Cardano
    "DOGEUSDT",  # Dogecoin
    "AVAXUSDT",  # Avalanche
    "LTCUSDT"    # Litecoin
]

execution_delay = [0, 1, 2, 3]
execution_cost_bps = [20, 30, 40]

stage_label = "001_download"
```

```
OUTPUT_ROOT = Path("../output")
OUTPUT_ROOT.mkdir(parents=True, exist_ok=True)
MANIFEST_FILE = OUTPUT_ROOT / "manifest.pkl"

DOWNLOAD_DIR = OUTPUT_ROOT / "001_download"
DOWNLOAD_DIR.mkdir(parents=True, exist_ok=True)

PRICES_FILE = DOWNLOAD_DIR / "PRICES.pkl"

inspection_window = 20

observation_window_length = 10
observation_window = range(1, observation_window_length + 1)

holding_period = 1

KLINE_INTERVAL = "1d"
```

3. Load or Initialize the Manifest

The manifest is a shared pickle dictionary that tracks artifact paths and timestamps across all pipeline stages. If it doesn't exist yet, it is initialized here.

In [3]:
```
if MANIFEST_FILE.exists():
    manifest = pd.read_pickle(MANIFEST_FILE)
else:
    manifest = {}

manifest.setdefault(stage_label, {})
```

Out[3]:
```
{}
```

4. Download Daily OHLCV Data

Fetches daily klines (open, high, low, close, volume) from Binance for each asset in the universe, starting from `startdate`. Results are stacked into a single long-format DataFrame and persisted to `PRICES.pkl`.

In [4]:
```
client = Client(tld="US")

if startdate is not None and str(startdate).strip():
    start_ts = pd.to_datetime(startdate, utc=True)
else:
    start_ts = pd.Timestamp.now(tz="UTC") - pd.Timedelta(days=365)

start_ms = int(start_ts.timestamp() * 1000)

price_frames = []

for symbol in universe:
    klines = client.get_historical_klines(
        symbol,
        KLINE_INTERVAL,
        start_ms
    )
    df = pd.DataFrame(
        klines,
        columns=[
            "open_time","open","high","low","close","volume",
            "close_time","quote_asset_volume","number_of_trades",
            "taker_buy_base_asset_volume","taker_buy_quote_asset_volume","ignore"
        ]
    )
    df["date"] = pd.to_datetime(df["open_time"], unit="ms", utc=True).dt.normalize()
    df[["open","close","volume"]] = df[
        ["open","close","volume"]
    ].astype(float)

    df = df[["date","open","close","volume"]]
    df["coin"] = symbol

    price_frames.append(df)

prices = pd.concat(price_frames, ignore_index=True)
prices = prices[["date","coin","open","close","volume"]]

prices.to_pickle(PRICES_FILE)

# --- manifest registration ---
manifest[stage_label]["prices"] = str(PRICES_FILE)

print("saved:", PRICES_FILE)
print("shape:", prices.shape)
```

```
saved: ..\output\001_download\PRICES.pkl
shape: (10984, 5)
```

5. Validate the Downloaded Price Data

Sanity checks on shape, date coverage, and head/tail rows to confirm the download completed correctly before proceeding.

In [5]:
```
print(prices.shape)
print(prices.head())
print(prices.tail())
```

```
(10984, 5)
                       date     coin      open     close       volume
0 2023-01-01 00:00:00+00:00  BTCUSDT  16524.41  16617.57   309.056051
1 2023-01-02 00:00:00+00:00  BTCUSDT  16619.93  16677.87   713.131373
2 2023-01-03 00:00:00+00:00  BTCUSDT  16674.84  16674.12   743.945758
3 2023-01-04 00:00:00+00:00  BTCUSDT  16676.49  16849.97  1511.331821
4 2023-01-05 00:00:00+00:00  BTCUSDT  16852.47  16832.48   722.090851
                          date     coin   open  close   volume
10979 2026-05-22 00:00:00+00:00  LTCUSDT  54.14  52.68  303.733
10980 2026-05-23 00:00:00+00:00  LTCUSDT  52.68  53.32  270.823
10981 2026-05-24 00:00:00+00:00  LTCUSDT  53.45  52.87  171.407
10982 2026-05-25 00:00:00+00:00  LTCUSDT  52.88  52.69  444.272
10983 2026-05-26 00:00:00+00:00  LTCUSDT  52.67  52.32  638.781
```

6. Construct Per-Asset Event Panels

For each reference coin, builds a panel where each row captures:

- The reference coin's close price on a given date
- The target coin's closing prices over the subsequent `observation_window_length` days

This panel structure is the foundation for measuring how follower assets respond to leader shocks in later stages.

In [6]:
```
prices = pd.read_pickle(PRICES_FILE)

close_matrix = prices.pivot(
    index="date",
    columns="coin",
    values="close"
).sort_index()

close_matrix = close_matrix.dropna(how="any")

for ref_coin in universe:

    rows = []

    for i in range(len(close_matrix) - observation_window_length):

        date = close_matrix.index[i]
        ref_close = close_matrix.iloc[i][ref_coin]

        if pd.isna(ref_close):
            continue

        for target_coin in universe:

            if target_coin == ref_coin:
                continue

            lag_closes = close_matrix[target_coin].iloc[
                i+1 : i+1+observation_window_length
            ].values.tolist()

            if any(pd.isna(lag_closes)):
                continue

            rows.append({
                "ref_coin": ref_coin,
                "date": date,
                "ref_close": ref_close,
                "target_coin": target_coin,
                "target_close_lags": lag_closes
            })

    panel = pd.DataFrame(rows)

    outfile = DOWNLOAD_DIR / f"{ref_coin}.event_panel.pkl"
    panel.to_pickle(outfile)

    # --- register artifact in manifest ---
    manifest[stage_label][ref_coin] = str(outfile)

    print(f"saved {outfile}  rows={len(panel)}")
```

```
saved ..\output\001_download\BTCUSDT.event_panel.pkl  rows=8304
saved ..\output\001_download\ETHUSDT.event_panel.pkl  rows=8304
saved ..\output\001_download\BNBUSDT.event_panel.pkl  rows=8304
saved ..\output\001_download\SOLUSDT.event_panel.pkl  rows=8304
saved ..\output\001_download\XRPUSDT.event_panel.pkl  rows=8304
saved ..\output\001_download\ADAUSDT.event_panel.pkl  rows=8304
saved ..\output\001_download\DOGEUSDT.event_panel.pkl  rows=8304
saved ..\output\001_download\AVAXUSDT.event_panel.pkl  rows=8304
saved ..\output\001_download\LTCUSDT.event_panel.pkl  rows=8304
```

7. Validate Event Panel Files

Checks that each per-coin pickle file has the correct schema, consistent lag lengths, no missing reference prices, and only known target coins. Fails fast with a descriptive assertion if anything is malformed.

In [7]:
```
expected_columns = [
    "ref_coin",
    "date",
    "ref_close",
    "target_coin",
    "target_close_lags"
]

for coin in universe:
```

```
    path = DOWNLOAD_DIR / f"{coin}.event_panel.pkl"

    print(f"Checking: {path}")

    df = pd.read_pickle(path)

    # column validation
    assert list(df.columns) == expected_columns, "Issue: Column mismatch"

    # ref coin check
    assert (df["ref_coin"] == coin).all(), "Issue: Reference coin mismatch"

    # target coin check
    invalid_targets = set(df["target_coin"]) - set(universe)
    assert len(invalid_targets) == 0, "Issue: Unknown target coins found"

    # lag length validation
    lag_lengths = df["target_close_lags"].apply(len).unique()
    assert len(lag_lengths) == 1, "Issue: Inconsistent lag lengths"
    assert lag_lengths[0] == observation_window_length, "Issue: Incorrect lag length"

    # basic sanity checks
    assert df["ref_close"].notna().all(), "Issue: Missing reference prices"

    print("Validation: Passed")
```

```
Checking: ..\output\001_download\BTCUSDT.event_panel.pkl
Validation: Passed
Checking: ..\output\001_download\ETHUSDT.event_panel.pkl
Validation: Passed
Checking: ..\output\001_download\BNBUSDT.event_panel.pkl
Validation: Passed
Checking: ..\output\001_download\SOLUSDT.event_panel.pkl
Validation: Passed
Checking: ..\output\001_download\XRPUSDT.event_panel.pkl
Validation: Passed
Checking: ..\output\001_download\ADAUSDT.event_panel.pkl
Validation: Passed
Checking: ..\output\001_download\DOGEUSDT.event_panel.pkl
Validation: Passed
Checking: ..\output\001_download\AVAXUSDT.event_panel.pkl
Validation: Passed
Checking: ..\output\001_download\LTCUSDT.event_panel.pkl
Validation: Passed
```

999. Persist the Manifest

Records artifact paths and a completion timestamp for this stage into the shared manifest. Downstream notebooks use the manifest to locate their required inputs.

In [8]:
```
manifest[stage_label]["timestamp"] = datetime.now(UTC).isoformat()
pd.to_pickle(manifest, MANIFEST_FILE)
print("manifest saved:", MANIFEST_FILE)
print("stages:", list(manifest.keys()))
print("artifacts in stage:", list(manifest[stage_label].keys()))
```

```
manifest saved: ..\output\manifest.pkl
stages: ['001_download']
artifacts in stage: ['prices', 'BTCUSDT', 'ETHUSDT', 'BNBUSDT', 'SOLUSDT', 'XRPUSDT', 'ADAUSDT', 'DOGEUSDT', 'AVAXUSDT', 'LTCUSDT', 'timestamp']
```

In [ ]:
```

```